desing qoura website through html and css

Great practice idea, Aamir 👷‍♂️
Let's design a **Quora-style website UI** using **pure HTML & CSS** (no JS) — clean, professional, and beginner-friendly.

🧩 Project Structure

Create two files:

```
quora.html
style.css
```

Copy code

📄 quora.html

html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Quora Clone</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
    <div class="logo">Quora</div>
    <nav>
        <a href="#">Home</a>
        <a href="#">Answer</a>
        <a href="#">Spaces</a>
```

```
            <a href="#">Notifications</a>
            <a href="#">Profile</a>
        </nav>
    </header>

    <main>

    <div class="left">
        <h3>Topics</h3>
        <p>Technology</p>
        <p>Programming</p>
        <p>Science</p>
        <p>Education</p>
    </div>

    <div class="center">
        <div class="question-box">
            <h4>What is HTML and CSS?</h4>
            <p>HTML is the structure of a webpage and CSS is used for styling it.</p>
            <button>Answer</button>
        </div>

        <div class="question-box">
            <h4>Why is JavaScript important?</h4>
            <p>JavaScript makes websites interactive and dynamic.</p>
            <button>Answer</button>
        </div>
    </div>

    <div class="right">
        <h3>Trending</h3>
        <p>Web Development</p>
        <p>AI & ML</p>
        <p>Data Science</p>
    </div>

    </main>
```

```
</body>
</html>
```

🎨 style.css

css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, Helvetica, sans-serif;
}

body {
    background: #f1f2f2;
}

/* Header */

header {
    background: white;
    padding: 15px 40px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 1px solid #ddd;
}

.logo {
    color: #b92b27;
    font-size: 26px;
    font-weight: bold;
}

nav a {
    margin-left: 20px;
    text-decoration: none;
```

```
    color: #333;
    font-weight: 600;
}

/* Layout */

main {
    display: flex;
    padding: 20px 40px;
    gap: 20px;
}

.left, .right {
    width: 20%;
    background: white;
    padding: 15px;
    border-radius: 6px;
}

.center {
    width: 60%;
}

/* Question Box */

.question-box {
    background: white;
    padding: 15px;
    border-radius: 6px;
    margin-bottom: 15px;
}

.question-box h4 {
    margin-bottom: 8px;
}

.question-box p {
    color: #555;
    margin-bottom: 10px;
}
```

```
.question-box button {
    padding: 6px 12px;
    border: none;
    background: #b92b27;
    color: white;
    border-radius: 4px;
    cursor: pointer;
}
```

🧪 How It Looks

- ✔ Top navbar
- ✔ 3-column layout (like Quora)
- ✔ Question cards
- ✔ Clean modern UI

🧠 Concepts You Practice Here

- Flexbox layout
- Page structuring
- Real-world UI design
- Box model
- Positioning and spacing

If you want, next I can:

- 🔹 Add **login page**
- 🔹 Make it **responsive**
- 🔹 Add **hover effects & animations**
- 🔹 Convert it into **React project**

Just tell me 💬

universal selector

Great — **Universal Selector (*)** is a very important CSS concept 👍
Let's understand it **deeply but simply**.

🌍 What is Universal Selector?

The **universal selector** is written as:

```css
*
```

Copy code

It selects **every element on the webpage**.

🧠 Why we use it?

Mostly used to:

- Remove default margin & padding
- Apply common styles
- Create a clean layout base

🧪 Basic Example

```css
* {
    margin: 0;
    padding: 0;
}
```

This removes default spacing from:

```css
```

```
body, h1, p, div, span, ul, li — everything
```

Copy code

🧱 Real Use Case (Layout Reset)

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

🔍 What each line means:

Property	Meaning
`margin: 0`	removes outside spacing
`padding: 0`	removes inside spacing
`box-sizing: border-box`	keeps width/height stable

Copy code

This is used in **almost every professional project**.

🧩 Apply Common Styles

```css
* {
    font-family: Arial;
    color: #333;
}
```

All text will follow this unless overridden.

Copy code

⚔️ Specificity Rule

Universal selector has the **lowest priority**.

```css
* { color: red; }
p { color: blue; }
```

Paragraph will be **blue**, not red.

Copy code

🚫 What NOT to do

Avoid heavy styles:

```css
* {
    background: red;  ❌
}
```

It applies to everything → messy & slow.

Copy code

👩‍💻 Interview Question

Q: Why do developers use `*` selector at the start of CSS?

Answer:

To reset default browser spacing and create consistent layout behavior.

🧪 Small Demo

```css
* {
    margin: 0;
    padding: 0;
}
```

```html
<h1>Hello</h1>
<p>World</p>
```

Without `*` → space around elements

With `*` → no space

Copy code

If you want, next I can explain:

- **Element selector**
- **Class vs ID selector**
- **Attribute selectors**
- **CSS Specificity system**

You're learning real frontend now, Aamir 💻🔥

element selector

Perfect 👍

Element Selector is one of the most basic and powerful CSS selectors.

Let's break it down clearly 👇

🧩 What is Element Selector?

An **element selector** selects HTML elements **by their tag name**.

🧪 Syntax:

```css
tagname {
    property: value;
```

```
}
```

🧱 Example:

Copy code

```css
p {
    color: blue;
}
```

This styles **all** `<p>` **elements** on the page.

Copy code

📌 Common Element Selectors

```css
h1 { }
p { }
div { }
span { }
img { }
a { }
button { }
```

🧠 How it Works

Copy code

HTML:

```html
<p>Hello</p>
<p>World</p>
```

Copy code

CSS:

```css
```

```css
p {
    color: red;
    font-size: 18px;
}
```

Copy code

Both paragraphs become red & 18px.

🧱 Multiple Elements at Once

css

```css
h1, h2, h3 {
    color: darkgreen;
}
```

Copy code

🪜 Element Selector + Descendant

css

```css
div p {
    color: purple;
}
```

This selects only `<p>` inside `<div>`.

Copy code

⚔️ Specificity Level

Selector	Priority
Universal `*`	Lowest
Element `p`	Low
Class `.box`	Medium
ID `#main`	High

Selector	Priority
Inline style	Highest

So this wins:

```css
p { color: blue; }
.text { color: red; }
```

If `<p class="text">` → color will be **red**.

Copy code

🚫 Common Mistake

```css
#p { }  ❌ (this is ID selector, not element)
.p { }  ❌ (this is class selector)
p { }   ✅ element selector
```

Copy code

👷 Real Use Case

```css
body {
    background: #f5f5f5;
}

a {
    text-decoration: none;
}

img {
    max-width: 100%;
}
```

🧪 Mini Practice

Question:

Make all headings blue and all paragraphs gray.

Copy code

```css
h1, h2, h3 {
    color: blue;
}

p {
    color: gray;
}
```

If you want, next we go to:

Copy code

➤ Class Selector vs ID Selector

(super important for interviews & real projects)

id selector

Great 👍

Now you're reaching the **most important CSS selector** for layout & JS control:

🆔 ID Selector

Let's understand it properly 👇

🧩 What is ID Selector?

An **ID selector** selects **one unique element** using its `id` attribute.

🧪 Syntax:

```css
#idName {
    property: value;
}
```

Copy code

🧱 Example

HTML:

```html
<h1 id="title">Welcome</h1>
<p>This is a paragraph</p>
```

Copy code

CSS:

```css
#title {
    color: red;
    font-size: 30px;
}
```

Only the `<h1>` becomes red & 30px.

Copy code

🧠 Key Rules

Rule	Explanation
ID must be **unique**	Only one element should have that ID
One element → one ID	You should not reuse the same ID
ID has **high priority**	Higher than class & element

⚔️ Specificity Order

css

```css
p { color: blue; }        /* Low */
.text { color: green; }  /* Medium */
#title { color: red; }   /* High */
```

Result: **red**

Copy code

🧪 Multiple Properties Example

css

```css
#loginBtn {
    background: #b92b27;
    color: white;
    padding: 10px 15px;
    border-radius: 5px;
}
```

🚫 Common Mistakes

Copy code

html

```html
<p id="box"></p>
<div id="box"></div> ❌ wrong (duplicate ID)
```

Copy code

👩‍💻 Real Use Case

Used when:

- ✔ Targeting main containers
- ✔ JS manipulation
- ✔ Important unique sections

html

```
<div id="header"></div>
<div id="sidebar"></div>
<div id="footer"></div>
```

Copy code

VS ID vs Class

Feature	ID	Class
Symbol	#	.
Unique	Yes	No
Priority	High	Medium
Reusable	❌	✔

🧪 Mini Practice

Style the login button:

html

```
<button id="login">Login</button>
```

css

```
#login {
    background: green;
    color: white;
    padding: 10px;
}
```

Copy code

Next concept you **must** learn:

➤ Class Selector

(after that: **attribute selector & pseudo-classes**)

class selector

Excellent — now you're at the **most used selector in real projects** 👌

🧱 Class Selector

🧩 What is Class Selector?

A **class selector** styles **one or many elements** using the `class` attribute.

🧪 Syntax:

```css
.className {
    property: value;
}
```

Copy code

🧱 Basic Example

HTML:

```html
<p class="text">Hello</p>
<p class="text">World</p>
```

Copy code

CSS:

```css
```

```css
.text {
    color: blue;
    font-size: 18px;
}
```

Copy code

Both paragraphs become blue & 18px.

🧠 Key Advantages

Feature	Class
Reusable	✔ Yes
Multiple per element	✔ Yes
Good for layouts	✔ Yes
Priority	Medium

🧩 One Element → Multiple Classes

html

```html
<div class="card shadow large"></div>
```

css

```css
.card { background: white; }
.shadow { box-shadow: 0 0 5px gray; }
.large { width: 300px; }
```

Copy code

⚔️ Priority Comparison

css

```
p { color: blue; }
.text { color: green; }
#title { color: red; }
```

Copy code

For:

```
html
<p class="text" id="title">Hello</p>
```

Copy code

Result → **Red** (ID wins)

👷 Real Use Case

```
css
.container { width: 80%; margin: auto; }
.btn { padding: 8px 15px; border-radius: 5px; }
.btn-primary { background: blue; color: white; }
```

Copy code

🚫 Common Mistakes

```
css
#text { } ❌ ID selector
text { } ❌ element selector
.text { } ✅ class selector
```

Copy code

🧪 Mini Practice

Make 3 boxes using one class:

```
html
<div class="box"></div>
<div class="box"></div>
```

```
<div class="box"></div>
```

css

```
.box {
    width: 100px;
    height: 100px;
    background: orange;
}
```

Copy code

VS Class vs ID

Feature	Class	ID
Symbol	.	#
Reuse	✔	✘
Specificity	Medium	High
JS targeting	Often	Often

Next powerful concepts you should learn:

➤ **Attribute Selector**

➤ **Pseudo-classes (`:hover` , `:focus`)**

➤ **Pseudo-elements (`::before` , `::after`)**

Descendant selector

Excellent — **Descendant Selector** is a core concept for real layouts & clean CSS 🔥
Let's understand it step-by-step.

🌳 What is Descendant Selector?

It selects an element **inside another element**, no matter how deep it is.

🧪 Syntax:

```css
parent child {
    property: value;
}
```

Copy code

It means:

> Select all **child elements inside parent**

🧱 Basic Example

HTML:

```html
<div>
    <p>Hello</p>
</div>

<p>World</p>
```

CSS:

```css
div p {
    color: red;
}
```

Result:

Copy code

- `Hello` → **Red**
- `World` → unchanged

Because only `<p>` inside `<div>` is selected.

🧩 Multi-level Descendant

```css
section div p {
    color: blue;
}
```

Selects `<p>` inside `<div>` inside `<section>`.

Copy code

🧠 Why Use It?

- ✔ Cleaner CSS
- ✔ Avoid unnecessary classes
- ✔ Target specific sections
- ✔ Essential for layout control

👩‍💻 Real Website Example

```css
header nav a {
    text-decoration: none;
    color: black;
}
```

Targets only links inside navbar.

Copy code

🪜 Difference: Descendant vs Child

Selector	Meaning
`div p`	Any `<p>` inside `<div>`
`div > p`	Only direct child `<p>`

🧪 Compare Example

html

```
<div>
   <p>One</p>
   <section>
      <p>Two</p>
   </section>
</div>
```

css

```
div p { color: red; }    /* One & Two */
div > p { color: blue; } /* Only One */
```

Copy code

⚔️ Specificity

`div p` = two element selectors → stronger than just `p`.

🧪 Mini Practice

Style only menu links:

html

```
<nav>
   <a>Home</a>
   <a>About</a>
</nav>
```

```
<a>Login</a>
```

```css
nav a {
    color: green;
}
```

Copy code

🧠 Interview Question

Q: What is a descendant selector?

Answer:

It selects elements that are inside another specified element.

Next concepts that connect with this:

➤ Child Selector (>)

➤ Adjacent Sibling (+)

➤ General Sibling (~)

You're building real CSS power now 💪

sibling selector

Great — **Sibling Selectors** are extremely important for layouts & UI behavior 🔥

There are **two types of sibling selectors** in CSS:

🧩 1. Adjacent Sibling Selector (+)

Selects the **very next** sibling element.

🧪 Syntax:

```css
element1 + element2 {
    property: value;
}
```

🧱 Example

Copy code

```html
<h2>Title</h2>
<p>First paragraph</p>
<p>Second paragraph</p>
```

```css
h2 + p {
    color: red;
}
```

Only **First paragraph** becomes red.

Copy code

🧩 2. General Sibling Selector (~)

Selects **all** matching siblings after the first element.

🧪 Syntax:

```css
element1 ~ element2 {
    property: value;
}
```

🧱 Example

Copy code

```css
h2 ~ p {
    color: blue;
}
```

Both paragraphs become blue.

Copy code

🧠 Key Difference

Selector	Affects
+	Only the **immediate next** element
~	**All following** siblings

🧪 Real UI Example

```html
<label>Email</label>
<input type="email">
<small>Error message</small>
```

```css
input:focus + small {
    color: red;
}
```

Shows error message only when input is focused.

Copy code

🧩 Another Example

```css
```

```
.menu h3 ~ p {
    color: gray;
}
```

Copy code

Styles all `<p>` after `<h3>` inside `.menu`.

⚔️ Specificity

Sibling selectors are **stronger than element selectors** but weaker than class & ID.

🧠 Interview Question

Q: Difference between `+` and `~`?

Answer:

`+` selects the immediate sibling,

`~` selects all following siblings.

🧪 Practice Challenge

html

```
<h1>Heading</h1>
<p>Para 1</p>
<p>Para 2</p>
```

Make only **Para 1** red:

Copy code

css

```
h1 + p { color: red; }
```

Copy code

Next powerful concept:

➤ Attribute Selectors

(Used heavily in forms & frameworks)

child selector

Excellent — **Child Selector** is one of the most important layout selectors in CSS 💪
It gives you **precise control** over elements.

👨‍👦 Child Selector (>)

Selects only the **direct children** of an element.

🧪 Syntax

```css
parent > child {
    property: value;
}
```

Copy code

🧱 Basic Example

HTML

```html
<div>
   <p>One</p>
   <section>
       <p>Two</p>
   </section>
</div>
```

CSS

```css
div > p {
    color: red;
}
```

Copy code

Result

- **One** → red
- **Two** → unchanged

Because only the **direct** `<p>` **child** is selected.

🧠 Child vs Descendant

Selector	Selects
`div p`	All `<p>` inside `<div>`
`div > p`	Only **direct child**

```css
nav > a {
    padding: 10px;
}
```

Only styles links that are directly inside `<nav>` .

Copy code

🧪 Another Example

```html
```

```
<ul>
   <li>Home</li>
   <li>
       About
       <ul>
           <li>Team</li>
       </ul>
   </li>
</ul>
```

css

```
ul > li {
    color: green;
}
```

Only first-level `<li>` items become green.

Copy code

⚔️ Specificity

More specific than descendant selector.

🧠 Interview Question

Q: Why use child selector instead of descendant selector?

Answer:

To avoid applying styles to deeply nested elements and maintain strict layout control.

🧪 Mini Practice

html

```
<div>
  <span>A</span>
  <p>B</p>
```

```
  <section>
    <span>C</span>
  </section>
</div>
```

Make only **A & B** blue:

```css
div > span, div > p {
    color: blue;
}
```

Copy code

Next topic to master:

➤ Attribute Selector

(super useful in forms, login pages, validations)

attribute selector

Perfect — **Attribute Selector** is extremely powerful in real projects & forms 🔥
Let's master it clearly.

🏷️ Attribute Selector

It selects elements based on the **presence or value of attributes**.

💉 1. Select by Attribute Presence

```css
```

```css
input[type] {
    border: 2px solid blue;
}
```

Copy code

Selects all inputs that have a `type` attribute.

🧪 2. Select by Exact Value

css

```css
input[type="text"] {
    background: lightyellow;
}
```

Only text inputs.

Copy code

🧪 3. Starts With `^`

css

```css
a[href^="https"] {
    color: green;
}
```

Links starting with `https` .

Copy code

🧪 4. Ends With `$`

css

```css
img[src$=".png"] {
    border: 2px solid red;
}
```

Only PNG images.

Copy code

🧪 5. Contains `*`

```css
a[href*="google"] {
    color: orange;
}
```

Any link containing "google".

Copy code

🧪 6. Space-separated Value `~`

```css
div[class~="box"] {
    background: lightblue;
}
```

Matches one whole word in class list.

Copy code

🧪 7. Hyphen-separated `|`

```css
p[lang|="en"] {
    color: blue;
}
```

Matches `en` or `en-US`.

Copy code

👩‍💻 Real Login Form Example

```css
input[type="password"] {
    border: 2px solid red;
}
```

```
input[required] {
    background: #f2f2f2;
}
```

⚔️ Specificity

Copy code

Same level as **class selector**.

🧠 Interview Question

Q: Why are attribute selectors important?

Answer:
They allow styling without adding extra classes and provide dynamic control over elements.

🧪 Mini Practice

html

```
<input type="text">
<input type="email">
<input type="password">
```

Make only email input green:

Copy code

css

```
input[type="email"] {
    background: lightgreen;
}
```

Copy code

Next powerful topics:

➤ **Pseudo-classes**

➤ **Pseudo-elements**

➤ **CSS Specificity & Cascade**

pseudo class

Excellent — **Pseudo-classes** are what make websites feel **interactive & alive** ⚡
This is a very important topic.

🎯 Pseudo-Class

A **pseudo-class** selects an element based on its **state**.

🧪 Syntax

```css
selector:pseudo-class {
    property: value;
}
```

Copy code

🧱 Most Important Pseudo-Classes

🖱️ :hover

When mouse is over element

```css
button:hover {
    background: green;
```

```
    color: white;
}
```

Copy code

🖱️ :active

When element is clicked

```css
button:active {
    transform: scale(0.95);
}
```

Copy code

🎯 :focus

When input is selected

```css
input:focus {
    border: 2px solid blue;
}
```

Copy code

🧪 :visited

After link is opened

```css
a:visited {
    color: purple;
}
```

Copy code

🧪 :checked

When checkbox/radio is selected

```css
input:checked {
    accent-color: green;
}
```

Copy code

:first-child

```css
li:first-child {
    color: red;
}
```

Copy code

:last-child

```css
li:last-child {
    color: blue;
}
```

Copy code

:nth-child()

```css
li:nth-child(2) {
    color: orange;
}
```

Copy code

UI Example (Login Form)

```css
input:focus {
    outline: none;
    border-color: #0077ff;
}

button:hover {
    background: #0055cc;
}
```

🧠 Interview Question

Copy code

Q: What is pseudo-class?

Answer:

A pseudo-class selects elements based on their state or position.

🧪 Mini Practice

```html
<a href="#">Link</a>
```

```css
a:hover { color: red; }
a:visited { color: green; }
```

Copy code

Next big concept:

➤ Pseudo-Elements

(`::before` , `::after` , etc.)

cascading and spacficity

Excellent topic 👌

Cascading & Specificity decide **which CSS rule actually works** when many rules target the same element.

Let's make this crystal clear 🧠

🌊 Cascading in CSS

Cascading means:

> When multiple CSS rules apply to the same element, CSS decides **which one wins**.

CSS follows this order:

🥇 1. Importance

```css
p { color: blue; }
p { color: red !important; }  → wins
```

Copy code

🥈 2. Specificity

More specific selector wins.

🥉 3. Source Order

Last written rule wins.

🧮 Specificity System

Each selector has a score:

Selector	Value
Inline style	1000
ID	100
Class, attribute, pseudo-class	10
Element, pseudo-element	1
Universal *	0

Example:

```css
#title .box p → 100 + 10 + 1 = 111
```

Copy code

⚔️ Who Wins?

```css
p { color: blue; }            /* 1 */
.text { color: green; }      /* 10 */
#para { color: red; }        /* 100 */
```

For:

Copy code

```html
<p id="para" class="text">Hello</p>
```

Copy code

👉 **Red wins**

🧱 When Specificity is Same → Last Rule Wins

```css
```

```css
p { color: blue; }
p { color: red; }
```

Copy code

👉 Red wins.

🚫 Avoid This Mistake

```css
#title { color: red !important; }  ❌ bad practice
```

Copy code

Use `!important` only as last option.

🧪 Real Case

```css
.nav a { color: blue; }     /* 11 */
header nav a { color: red; } /* 3 */
```

`.nav a` wins because **11 > 3**

Copy code

🧠 Interview Question

Q: Difference between cascading and specificity?

Answer:
Cascading decides which rule applies; specificity is the scoring system that helps decide that.

🧪 Quick Practice

Who wins?

```css
```

```
div p { color: blue; }     /* 2 */
p.text { color: green; }  /* 11 */
```

👉 **Green wins**

Copy code

Now you officially understand the **core of CSS** 🔥
If you want, next I can teach you:

➡️ Box Model
➡️ Display & Position
➡️ Flexbox
➡️ Grid

important in css

Great — `!important` is **small but extremely powerful** in CSS ⚠️
And also **dangerous if misused**.

Let's understand it properly.

❗ `!important` in CSS

`!important` forces a rule to **override everything else**, even high-specificity selectors.

🧪 Syntax

```css
selector {
    property: value !important;
}
```

Copy code

🧱 Example

```css
p { color: blue; }
#para { color: red; }

p { color: green !important; }
```

For:

```html
<p id="para">Hello</p>
```

Copy code

👉 Result: **Green**

Because `!important` beats ID.

🧮 Priority Order

1. `!important`
2. Inline styles
3. ID
4. Class / Attribute / Pseudo-class
5. Element
6. Universal

If **both** have `!important`, then **specificity** is used.

🧪 When both have `!important`

```css
p { color: blue !important; }
#para { color: red !important; }
```

👉 **Red** wins (ID has higher specificity).

Copy code

🧠 When should you use it?

- ✔ Fix third-party library CSS
- ✔ Override browser default styles
- ✔ Temporary debugging

🚫 When NOT to use it

- ❌ Regular styling
- ❌ Layout design
- ❌ Large projects

Overusing it makes CSS **unmaintainable**.

👩‍💻 Real Example

```css
button {
   background: gray;
}

.bootstrap-btn {
   background: blue !important;
}
```

🧠 Interview Question

Copy code

Q: Why is `!important` discouraged?

Answer:

Because it breaks the natural CSS hierarchy and makes code hard to maintain.

🧪 Mini Practice

```css
p { color: red; }
.text { color: blue !important; }
```

For:

Copy code

```html
<p class="text">Test</p>
```

Copy code

👉 **Blue**

You now know **how CSS really decides everything** 🧠🔥
Next topics that make you pro:

➡️ Box Model
➡️ Display Property
➡️ Positioning
➡️ Flexbox & Grid

inheritance

Excellent — **Inheritance in CSS** is one of the core principles that makes CSS powerful and efficient 🧠🔥
Let's understand it clearly.

🌱 Inheritance in CSS

Inheritance means:

Some CSS properties applied to a parent element are automatically passed down to its children.

🧪 Basic Example

css

```css
div {
    color: blue;
}
```

html

```html
<div>
   <p>Hello</p>
   <span>World</span>
</div>
```

Both **Hello** and **World** become blue because `color` is **inherited**.

Copy code

🧠 Properties That Inherit Naturally

Property	Inherits
`color`	✔
`font-family`	✔
`font-size`	✔
`visibility`	✔
`cursor`	✔
`line-height`	✔

🚫 Properties That Do NOT Inherit

Property	Inherits
`margin`	❌
`padding`	❌
`border`	❌
`background`	❌
`width/height`	❌

🧩 Force Inheritance

```css
button {
    color: inherit;
}
```

Forces button text to inherit from parent.

Copy code

🧪 Stop Inheritance

```css
p {
    color: initial;
}
```

or

Copy code

```css
p {
    color: unset;
```

```
}
```

Copy code

🧠 Real UI Example

```css
body {
    font-family: Arial;
    color: #333;
}
```

All text on the website inherits this automatically.

Copy code

⚔️ Inheritance vs Specificity

If a child has its own rule, it overrides inherited value.

```css
div { color: red; }
p { color: blue; }
```

`p` becomes blue.

Copy code

🧠 Interview Question

Q: What is inheritance in CSS?

Answer:

It is the process by which some CSS properties are passed from parent to child elements.

🧪 Mini Practice

```css
```

Copy code

```
section { font-size: 20px; }
```

Everything inside `<section>` will automatically become 20px unless overridden.

You now understand one of the **core engines of CSS** ⚙️
Next up (very important):

➡️ **Box Model**

explain sibling

Sure, Aamir 🙂
Let's explain **Sibling Selectors in CSS** very clearly and simply.

👫 What are Sibling Selectors?

Sibling selectors select elements that **share the same parent**.

There are **two types**:

1. **Adjacent Sibling Selector (+)**
2. **General Sibling Selector (~)**

🧩 1. Adjacent Sibling Selector (+)

Selects **only the very next sibling**.

Syntax

```
css
```

```
element1 + element2 {
   property: value;
}
```

Copy code

Example

html

```
<h2>Title</h2>
<p>First</p>
<p>Second</p>
```

css

```
h2 + p {
   color: red;
}
```

Result

Copy code

- **First** → red
- **Second** → unchanged

Because only the **first** `<p>` **immediately after** `<h2>` is selected.

🧩 2. General Sibling Selector (~)

Selects **all matching siblings that come after** the first element.

Syntax

css

```
element1 ~ element2 {
   property: value;
}
```

Example

Copy code

css

```
h2 ~ p {
   color: blue;
}
```

Result

Copy code

Both **First** and **Second** become blue.

🧠 Key Difference

Selector	Selects
+	Only the **next** element
~	**All following** elements

🧪 Real Website Example

html

Copy code

```
<input>
<small>Error message</small>
```

css

Copy code

```
input:focus + small {
   color: red;
}
```

Error message becomes red **only when input is focused**.

🧠 Interview Question

Q: What is the difference between `+` and `~`?

Answer:

+ selects the immediate next sibling,
~ selects all following siblings.

🧪 Simple Practice

html Copy code

```
<h1>Heading</h1>
<p>One</p>
<p>Two</p>
```

Make only **One** red:

css Copy code

```
h1 + p { color: red; }
```

If you'd like, next I can explain:

➡️ **Child vs Descendant vs Sibling selectors comparison**
➡️ **Combinators in CSS (important interview topic)**